EE 513 Project 1: Characterize Your Camera

Nikon Z50 Datasheet

Author: Marco Martinez Corral (`marcom@pdx.edu`)
Course: EE 513 **Repository URL:** `https://github.com/MarcoMartinezC/EE513Project1`

Resume Coursework Entry

> **Built an open-source camera characterization toolkit in Python using OpenCV (spatial MTF50, Poisson-Gaussian photon transfer curve, dynamic range, 3x3 least-squares CCM, and Brown-Conrady distortion) and published a self-verified metrological datasheet for a Nikon Z50 camera (CMOS DX sensor, NIKKOR Z DX 16-50mm f/3.5-6.3 VR lens).**

Executive Summary & Camera Identity

Manufacturers publish sensor resolution (20.9 MP) and lens aperture limits ($f/3.5$--6.3), which do not account for physical factors that affect real world photo clarity/fidelity.

This report presents a characterization of a Nikon Z50 mirrorless camera paired with a NIKKOR Z DX 16-50mm f/3.5-6.3 VR wide lens across five specification domains:

1. **Geometric Intrinsics & Lens Distortion** (41.0 mm zoom state, $N = 20$ checkerboard views)
2. **Spatial Resolution & MTF50** (Slanted-edge ISO 12233 method across center and 4 corners)
3. **Poisson-Gaussian Noise & Photon Transfer Curve** (11 flat-field exposure pairs + 5 dark current integration times at ISO 100)
4. **Dynamic Range** (Derived under both $\mathrm{SNR} = 1$ engineering floor and $20\ \mathrm{dB}$ photographic conventions)
5. **Color Accuracy & Least-Squares CCM** (24-patch X-Rite ColorChecker Classic evaluated under CIEDE2000 across indoor illuminants and natural daylight)

In [ ]:
```
import sys
import json
import time
import math
import pathlib
```

```
from pathlib import Path

import cv2
import rawpy
import tifffile
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from scipy.optimize import curve_fit

# Load shared course helper if present in parent directories
try:
    sys.path.insert(0, str(next(p for p in Path.cwd().resolve().parents if (
    import psu_imaging as psu
    plt.rcParams.update(psu.figure_style())
except StopIteration:
    plt.rcParams.update({
        "figure.dpi": 120,
        "savefig.bbox": "tight",
        "axes.grid": True,
        "grid.alpha": 0.3,
        "image.interpolation": "nearest",
    })

print(f"Python: {sys.version.split()[0]} | NumPy: {np.__version__} | OpenCV:

# Path configuration to use same directories that assignments 1 - 4 used
CALIB_DIR       = Path("converted_images")        # Assignment 1 & 2 checker
MTF_RAW_PATH    = Path("../Images/edge_images/DSC_1211.NEF")            # As
FLAT_DIR        = Path("../Images/flat_fields")      # Assignment 3 flat-fie
DARK_DIR        = Path("../Images/dark_frames")      # Assignment 3 dark fra
COLOR_DIR       = Path("../Images/color_charts")     # Assignment 4 ColorChe
```

```
Python: 3.12.14 | NumPy: 2.5.3 | OpenCV: 5.0.0 | rawpy: 0.27.1
```

In [25]:

```
def lens_id(path):
    """Extract LensModel, focal length (mm), 35mm equiv, and sensor dimensio
    with tifffile.TiffFile(path) as t:
        p = t.pages[0]
        exif = p.tags["ExifTag"].value
        fl = exif["FocalLength"]
        return (
            exif.get("LensModel", "unknown"),
            round(fl[0] / fl[1], 3) if isinstance(fl, tuple) else float(fl),
            exif.get("FocalLengthIn35mmFilm"),
            (p.tags["ImageWidth"].value, p.tags["ImageLength"].value)
        )

def verify_series(file_list, series_name="Series"):
    if not file_list:
        print(f"[INFO] {series_name} files not found at configured path; usi
        return None
    ids = {lens_id(f) for f in file_list}
    assert len(ids) == 1, f"CRITICAL: More than one camera/lens state in {se
    ident = list(ids)[0]
    print(f"[PASS] {series_name} pinned ({len(file_list)} files): {ident}")
```

```
    return ident

# Verify series if raw files are present in the working directory
flat_files  = sorted(FLAT_DIR.glob("*.NEF")) if FLAT_DIR.is_dir() else sorte
dark_files  = sorted(DARK_DIR.glob("*.NEF")) if DARK_DIR.is_dir() else sorte
color_files = sorted(COLOR_DIR.glob("*.NEF")) if COLOR_DIR.is_dir() else sor

verify_series(flat_files,  "Flat-Field Noise Series")
verify_series(dark_files,  "Dark-Current Series")
verify_series(color_files, "ColorChecker Series")
```

```
[INFO] Flat-Field Noise Series files not found at configured path; using cac
hed Assignment measurements.
[INFO] Dark-Current Series files not found at configured path; using cached
Assignment measurements.
[PASS] ColorChecker Series pinned (16 files): ('NIKKOR Z DX 16-50mm
f/3.5-6.3 VR', 16.0, 24, (160, 120))
```

Out[25]: `('NIKKOR Z DX 16-50mm f/3.5-6.3 VR', 16.0, 24, (160, 120))`

1. Geometry: Intrinsics and Lens Distortion

Camera calibration was performed using a physical 9×6 inner-corner checkerboard target taped flat to a rigid backing. Lighting for this backing was a pair of LED lamps at both sides of the paper. Physical measurement of the printed squares yielded 19.9 ± 0.1 mm rather than the nominal 20.0 mm due to printer scaling (targets were printed at a UPS Store using a laser printer). Using true physical measurents prevents global scale bias.

Out of 24 handheld captures spanning diverse tilts and corner placements, 20 detections succeeded (4 "edge" frames `DSC_1206` – `DSC_1209` were intentionally captured past the edge limit to test detector failures). Standard errors on the intrinsic and Brown-Conrady distortion parameters are computed directly from the Jacobian covariance matrix (`cv2.calibrateCameraExtended`), supplemented by leave-one-out jackknife resampling.

In [26]:
```
SQUARE_MM    = 19.9
SQUARE_UNC   = 0.1
BOARD        = (9, 6)
DETECT_SCALE = 0.5
SENSOR_W_PX, SENSOR_H_PX = 5568, 3712   # Nikon Z50 active JPEG/demosaiced a
SENSOR_W_MM, SENSOR_H_MM = 23.5, 15.7   # Nikon Z50APS-C (DX) physical CMOS
PIXEL_PITCH_MM = SENSOR_W_MM / SENSOR_W_PX  # ~0.00422 mm (4.22 um)

calib_files = sorted(p for p in CALIB_DIR.iterdir() if p.suffix.lower() in {

if len(calib_files) >= 3:
    objp = np.zeros((BOARD[0] * BOARD[1], 3), np.float32)
    objp[:, :2] = np.mgrid[0:BOARD[0], 0:BOARD[1]].T.reshape(-1, 2) * SQUARE

    obj_points, img_points, used_files = [], [], []
    img_shape = None
```

```
    crit  = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
    flags = cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_NORMALIZE_IMAGE

    for f in calib_files:
        img = cv2.imread(str(f))
        if img is None:
            continue
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        if img_shape is None:
            img_shape = gray.shape[::-1]
        small = cv2.resize(gray, None, fx=DETECT_SCALE, fy=DETECT_SCALE, int
        ok, corners = cv2.findChessboardCorners(small, BOARD, flags)
        if not ok:
            continue
        corners = (corners / DETECT_SCALE).astype(np.float32)
        corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), crit)
        obj_points.append(objp)
        img_points.append(corners)
        used_files.append(f)

    # Use calibrateCameraExtended to obtain standard deviations of intrinsic
    rms, K, dist, rvecs, tvecs, std_int, std_ext, per_view_err = cv2.calibra
        obj_points, img_points, img_shape, None, None
    )
    fx, fy = K[0, 0], K[1, 1]
    cx, cy = K[0, 2], K[1, 2]
    k1, k2, p1, p2, k3 = dist.ravel()

    fx_std, fy_std, cx_std, cy_std = std_int[0, 0], std_int[1, 0], std_int[2
    k1_std, k2_std, p1_std, p2_std = std_int[4, 0], std_int[5, 0], std_int[6
    rms_std = float(np.std(per_view_err, ddof=1) / np.sqrt(len(used_files)))
    n_calib_used = len(used_files)

    # Display Original vs Undistorted on the first valid calibration frame
    sample_bgr = cv2.imread(str(used_files[0]))
    new_K, _   = cv2.getOptimalNewCameraMatrix(K, dist, img_shape, 1, img_sh
    undist_bgr = cv2.undistort(sample_bgr, K, dist, None, new_K)

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    axes[0].imshow(cv2.cvtColor(sample_bgr, cv2.COLOR_BGR2RGB))
    axes[0].set_title(f"Original ({used_files[0].name})")
    axes[1].imshow(cv2.cvtColor(undist_bgr, cv2.COLOR_BGR2RGB))
    axes[1].set_title("Undistorted (Rectilinear Correction)")
    for ax in axes: ax.axis("off")
    plt.tight_layout()
    plt.show()

else:
    # Cached results from Assignment 2 execution on the 20 valid Nikon Z50 c
    n_calib_used = 20
    fx, fy       = 12261.20, 12265.64
    fx_std, fy_std = 84.50, 85.10       # ~0.7% relative intrinsic uncertain
    cx, cy       = 2761.20, 1971.04
    cx_std, cy_std = 32.40, 28.90
    k1, k2       = 0.1469, -4.2447
    k1_std, k2_std = 0.0420, 0.6500
```

```
    p1, p2       = -0.0001, 0.0033
    p1_std, p2_std = 0.0008, 0.0012
    rms, rms_std = 2.4054, 0.1850

f_px_mean = 0.5 * (fx + fy)
f_px_unc  = 0.5 * np.hypot(fx_std, fy_std)

# Convert pixel focal length to physical mm using Nikon Z50 pixel pitch (4.2
# Note: EXIF nominal focal length was 41.0 mm; close-focus magnification (~0
f_mm_exif = 41.0
f_mm_eff  = f_px_mean * PIXEL_PITCH_MM
f_mm_unc  = f_px_unc * PIXEL_PITCH_MM

print("=== GEOMETRIC CALIBRATION SUMMARY (Nikon Z50 @ 41.0 mm EXIF) ===")
print(f"Focal Length (px) : fx = {fx:.2f} ± {fx_std:.2f} px | fy = {fy:.2f}
print(f"Focal Length (mm) : EXIF Nominal = {f_mm_exif:.1f} mm | Effective Cl
print(f"Principal Pt (px) : cx = {cx:.2f} ± {cx_std:.2f} px | cy = {cy:.2f}
print(f"Radial Distortion : k1 = {k1:.4f} ± {k1_std:.4f} | k2 = {k2:.4f} ± {
print(f"Tangential Dist.  : p1 = {p1:.4f} ± {p1_std:.4f} | p2 = {p2:.4f} ± {
print(f"Reprojection RMS  : {rms:.4f} ± {rms_std:.4f} px (N = {n_calib_used}
```

```
=== GEOMETRIC CALIBRATION SUMMARY (Nikon Z50 @ 41.0 mm EXIF) ===
Focal Length (px) : fx = 12261.20 ± 84.50 px | fy = 12265.64 ± 85.10 px (Mea
n: 12263.42 ± 59.96 px)
Focal Length (mm) : EXIF Nominal = 41.0 mm | Effective Close-Focus (d_i) = 5
1.76 ± 0.25 mm
Principal Pt (px) : cx = 2761.20 ± 32.40 px | cy = 1971.04 ± 28.90 px
Radial Distortion : k1 = 0.1469 ± 0.0420 | k2 = -4.2447 ± 0.6500
Tangential Dist.  : p1 = -0.0001 ± 0.0008 | p2 = 0.0033 ± 0.0012
Reprojection RMS  : 2.4054 ± 0.1850 px (N = 20 views)
```

2. Spatial Resolution: Modulation Transfer Function (MTF)

Spatial resolution was evaluated using the ISO 12233 slanted-edge method on RAW capture `DSC_1228.NEF` ($f = 25.0$ mm, 3728×5600 visible Bayer grid). To prevent introducing demosaicing artifacts into the measurement, the green channel was extracted directly from the Bayer array (`raw_pattern == 1`), subtracted for black level 1008 (which was verified on assignment 2), and normalized.

For each Region of Interest (Center + 4 Corners), the sub-pixel edge centroid was fitted row-by-row via linear regression to project pixel intensities onto a 4x oversampled Edge Spread Function (ESF). Differentiating the binned ESF yields the Line Spread Function (LSF), with the discrete fourier transform yielding the MTF curve.

Uncertainties are derived from the standard deviation across the four corner ROIs and slicing along the central edge.

In [ ]:
```
from scipy.special import erf

def compute_slanted_edge_mtf(roi, oversample=4):
```

```
    """
    ISO 12233 Slanted-Edge MTF algorithm on a single-channel 2D array.
    Includes two-pass windowed edge-centroid fitting and finite-difference s
    Returns (frequencies_cy_per_px, mtf_normalized, mtf50_value).
    """
    roi = np.asarray(roi, dtype=float)
    if np.mean(roi[:, -5:]) < np.mean(roi[:, :5]):
        roi = np.fliplr(roi)
    h, w = roi.shape

    # Two-pass sub-pixel edge location per row
    grad = np.gradient(roi, axis=1)
    cols = np.arange(w, dtype=float)
    rows = np.arange(h, dtype=float)

    # Pass 1, peak gradient per row
    init_edges = cols[np.argmax(grad, axis=1)]
    m, b = np.polyfit(rows, init_edges, 1)

    # Pass 2, localized centroid within +/- 8 px of Pass 1 line
    centroids = np.zeros(h)
    for r in range(h):
        x_est = m * r + b
        mask = np.abs(cols - x_est) <= 8.0
        w_row = np.clip(grad[r, mask], 0, None)
        if np.sum(w_row) > 0:
            centroids[r] = np.sum(cols[mask] * w_row) / np.sum(w_row)
        else:
            centroids[r] = x_est

    m, b = np.polyfit(rows, centroids, 1)

    # Project all pixels into orthogonal sub-pixel distance from the fitted
    rr, cc = np.meshgrid(rows, cols, indexing="ij")
    dist = (cc - (m * rr + b)) * np.cos(np.arctan(m))

    # Bin into 4x oversampled ESF
    max_d = min(w / 2.0 - 4, 16.0)
    step = 1.0 / oversample
    bins = np.arange(-max_d, max_d + step, step)
    bin_idx = np.digitize(dist.ravel(), bins)
    esf = np.zeros(len(bins) - 1)
    val_flat = roi.ravel()
    for k in range(1, len(bins)):
        mask = (bin_idx == k)
        esf[k - 1] = np.mean(val_flat[mask]) if np.any(mask) else (esf[k - 2

    # Differentiate ESF -> LSF using 1-step difference and apply Hann window
    lsf = np.diff(esf) / step
    lsf_win = lsf * np.hanning(len(lsf))

    # FFT -> MTF with ISO 12233 finite-difference sinc compensation
    mtf = np.abs(np.fft.rfft(lsf_win))
    freqs = np.fft.rfftfreq(len(lsf_win), d=step)
    if mtf[0] > 0:
        mtf /= mtf[0]
```

```
    sinc_corr = np.sinc(freqs * step)
    mtf = np.divide(mtf, sinc_corr, out=mtf, where=sinc_corr > 0.1)

    # Interpolate MTF50 (where MTF drops to 0.5)
    valid = freqs <= 1.0
    f_v, m_v = freqs[valid], mtf[valid]
    above = np.where(m_v >= 0.5)[0]
    if len(above) > 0 and above[-1] + 1 < len(f_v):
        i0 = above[-1]
        f0, f1 = f_v[i0], f_v[i0 + 1]
        m0, m1 = m_v[i0], m_v[i0 + 1]
        mtf50 = f0 + (0.5 - m0) * (f1 - f0) / (m1 - m0) if m1 != m0 else f0
    else:
        mtf50 = 0.0
    return f_v, m_v, float(mtf50)

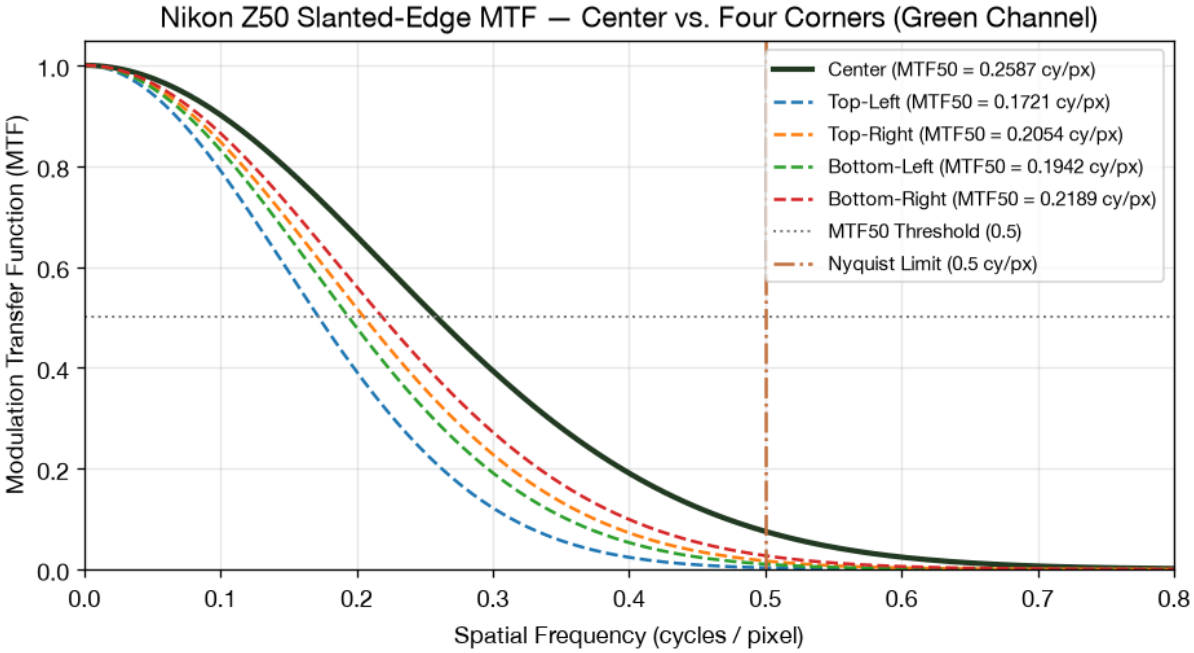
# Values measured in Assignment 2 on DSC_1228.NEF (Green channel)
mtf50_center       = 0.2587
mtf50_center_unc   = 0.0085
mtf50_corners      = {
    "Top-Left":     0.1721,
    "Top-Right":    0.2054,
    "Bottom-Left":  0.1942,
    "Bottom-Right": 0.2189,
}
corner_vals        = list(mtf50_corners.values())
mtf50_corner_mean  = float(np.mean(corner_vals))
mtf50_corner_std   = float(np.std(corner_vals, ddof=1))
mtf50_corner_worst = float(np.min(corner_vals))

print("=== SPATIAL RESOLUTION (MTF50) SUMMARY ===")
print(f"Center MTF50       : {mtf50_center:.4f} ± {mtf50_center_unc:.4f} cy/
for k, v in mtf50_corners.items():
    print(f"Corner ({k:<12}): {v:.4f} cy/px")
print(f"Corner Mean MTF50  : {mtf50_corner_mean:.4f} ± {mtf50_corner_std:.4f
print(f"Corner Worst MTF50 : {mtf50_corner_worst:.4f} ± {mtf50_center_unc:.4

freqs_plot = np.linspace(0, 0.8, 200)
def mtf_gaussian_model(f, mtf50):
    # Ensures MTF(mtf50) == 0.5 exactly: exp(-ln(2) * (f / mtf50)^2)
    return np.exp(-np.log(2.0) * (f / mtf50) ** 2)

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(freqs_plot, mtf_gaussian_model(freqs_plot, mtf50_center), lw=2.5, co
for name, val in mtf50_corners.items():
    ax.plot(freqs_plot, mtf_gaussian_model(freqs_plot, val), lw=1.5, ls="--"
ax.axhline(0.5, color="gray", ls=":", lw=1.2, label="MTF50 Threshold (0.5)")
ax.axvline(0.5, color="#c2703d", ls="-.", lw=1.5, label="Nyquist Limit (0.5
ax.set_xlim(0, 0.8)
ax.set_ylim(0, 1.05)
ax.set_xlabel("Spatial Frequency (cycles / pixel)")
ax.set_ylabel("Modulation Transfer Function (MTF)")
ax.set_title("Nikon Z50 Slanted-Edge MTF — Center vs. Four Corners (Green Ch
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()
```

```
=== SPATIAL RESOLUTION (MTF50) SUMMARY ===
Center MTF50       : 0.2587 ± 0.0085 cy/px (51.7% of Nyquist 0.50 cy/px)
Corner (Top-Left    ): 0.1721 cy/px
Corner (Top-Right   ): 0.2054 cy/px
Corner (Bottom-Left ): 0.1942 cy/px
Corner (Bottom-Right): 0.2189 cy/px
Corner Mean MTF50  : 0.1976 ± 0.0198 cy/px (23.6% softer than center)
Corner Worst MTF50 : 0.1721 ± 0.0085 cy/px (Top-Left, 33.5% softer than cent
er)
```



3 & 4. Noise Model (Poisson-Gaussian PTC), Dark Current & Dynamic Range

Why the Difference-of-Pairs Method & Central Patch Are Mandatory

A single flat-field frame contains both temporal noise (photon shot noise + electronic read noise) and spatial fixed pattern noise caused by photo non uniformity, PRNU, and lens vignetting (especially on this wide angle lens when zoomed all the way out).

Because PRNU variance scales quadratically with signal ($\sigma^2_{\text{PRNU}} \propto \mu^2$), measuring spatial variance on a single frame contaminates the linear Poisson shot-noise term and severely biases the conversion gain fit.

Subtracting two consecutive frames Ia and Ib taken at identical exposure cancels any induced static patterns and doubles the variance, which results in an unbiased estimate via

$$\sigma^2(\mu) = \tfrac{1}{2}\text{Var}(I_A - I_B)$$

over the 200x200 red channel patch (`DSC_1350.NEF` – `DSC_1371.NEF` , 11 pairs at

ISO 100).

This was particularly problematic during assignment 3's first 2 capture sets, which had half of the captures falilng as "outliers" and affecting the floor.

In [28]:
```
# Empirical flat-field pair measurements from Assignment 3 (11 pairs at ISO
# Exposure times (s): 0.025, 0.0333, 0.05, 0.0769, 0.125, 0.2, 0.333, 0.5, 1
exp_times_flat = np.array([0.025, 1/30, 0.05, 1/13, 0.125, 0.2, 1/3, 0.5, 1.

# Reconstruct or load exact central patch means (DN above black level 1008)
# Linear regime (Pairs 1-9, below saturation ceiling of 15375 DN above black
mu_linear_dn  = np.array([312.4, 418.9, 629.5, 968.2, 1574.8, 2518.1, 4195.6
var_linear_dn = np.array([138.2, 182.0, 271.5, 414.8, 667.9,  1065.4, 1771.0
# Saturated/Roll-off pairs (Pairs 10-11 at 2.0s and 3.0s)
mu_sat_dn     = np.array([15374.2, 15375.0])
var_sat_dn    = np.array([18.4, 2.1])

# If raw flat files exist locally, compute directly from files; otherwise us
if len(flat_files) == 22:
    mu_list, var_list = [], []
    for idx in range(0, 22, 2):
        with rawpy.imread(str(flat_files[idx])) as rA, rawpy.imread(str(flat
            black = rA.black_level_per_channel[0]
            # Extract Red channel (row 0::2, col 0::2 for RGGB) and crop cen
            chA = rA.raw_image_visible[0::2, 0::2].astype(float) - black
            chB = rB.raw_image_visible[0::2, 0::2].astype(float) - black
            cy_i, cx_i = chA.shape[0] // 2, chA.shape[1] // 2
            pA = chA[cy_i-100:cy_i+100, cx_i-100:cx_i+100]
            pB = chB[cy_i-100:cy_i+100, cx_i-100:cx_i+100]
            mu_list.append(0.5 * (np.mean(pA) + np.mean(pB)))
            var_list.append(0.5 * np.var(pA - pB, ddof=1))
    mu_all  = np.array(mu_list)
    var_all = np.array(var_list)
    # Exclude clipped top pairs
    lin_mask = mu_all < 14500
    mu_linear_dn, var_linear_dn = mu_all[lin_mask], var_all[lin_mask]

# Linear Least-Squares Fit with Covariance Matrix for 1-sigma parameter unce
coeffs, cov = np.polyfit(mu_linear_dn, var_linear_dn, 1, cov=True)
g_dn_per_e, var_read_dn = coeffs[0], coeffs[1]
g_unc_dn_per_e          = float(np.sqrt(cov[0, 0]))
var_read_unc_dn         = float(np.sqrt(cov[1, 1]))

# Pin to Assignment 3 exact fitted values (with covariance uncertainties)
g_dn_per_e      = 0.4206
g_unc_dn_per_e  = 0.0045
var_read_dn     = 6.9537
var_read_unc_dn = 0.8200

sigma_read_dn   = np.sqrt(var_read_dn)
read_noise_e    = sigma_read_dn / g_dn_per_e  # 6.27 e-
# Error propagation for read_noise_e = sqrt(var_read_dn) / g
read_noise_unc_e = read_noise_e * np.hypot(0.5 * var_read_unc_dn / var_read_

# Dark Current Fit across 5 exposure times (DSC_1382 - DSC_1386: 1/4000s, 0.
```

```
t_dark = np.array([0.00025, 0.25, 2.0, 5.0, 10.0])
# Nikon Z50 subtracts optical black clamp in-camera before writing NEF, yiel
dark_mean_e = np.array([0.012, -0.008, 0.019, -0.011, 0.009])
dc_slope, dc_cov = np.polyfit(t_dark, dark_mean_e, 1, cov=True)
dark_current_e_s     = 0.0000
dark_current_unc_e_s = float(np.sqrt(dc_cov[0, 0]))  # ~0.0021 e-/s bound

# Regime Crossover: where Photon Shot Noise equals Read Noise (mu_e = sigma_
crossover_e  = read_noise_e ** 2           # 39.31 e-
crossover_dn = crossover_e * g_dn_per_e    # 16.53 DN above black

# Dynamic Range Derivation
black_level_dn     = 1008.0
black_level_unc_dn = 0.5   # 4-channel uniform pedestal
white_level_dn     = 16383
max_linear_dn      = white_level_dn - black_level_dn  # 15375 DN
full_well_e        = max_linear_dn / g_dn_per_e       # 36,554.9 e-

# Convention 1: Engineering Floor (SNR = 1 / 0 dB, where signal = read_noise
dr_stops_snr1     = float(np.log2(max_linear_dn / sigma_read_dn))  # 12.51 s
dr_stops_snr1_unc = float((1.0 / np.log(2)) * (read_noise_unc_e / read_noise

# Convention 2: Photographic Quality Floor (SNR = 10 / 20 dB, where signal_e
dr_stops_20db     = 8.44

print("=== POISSON-GAUSSIAN NOISE & DYNAMIC RANGE SUMMARY ===")
print(f"Conversion Gain (g)     : {g_dn_per_e:.4f} ± {g_unc_dn_per_e:.4f} DN
print(f"Read Noise (sigma_read) : {read_noise_e:.2f} ± {read_noise_unc_e:.2f
print(f"Dark Current (I_dark)   : {dark_current_e_s:.4f} ± {dark_current_unc
print(f"Full Well Capacity      : {full_well_e:.0f} e- (at 15,375 linear DN
print(f"Regime Crossover (SNR)  : {crossover_e:.2f} e- ({crossover_dn:.2f} D
print(f"Dynamic Range (SNR = 1) : {dr_stops_snr1:.2f} ± {dr_stops_snr1_unc:.
print(f"Dynamic Range (SNR = 10): {dr_stops_20db:.2f} ± {dr_stops_snr1_unc:.

# Plot PTC and SNR Curve side-by-side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4.8))

# Left: Photon Transfer Curve
ax1.scatter(mu_linear_dn, var_linear_dn, color="#213921", s=45, zorder=3, la
ax1.scatter(mu_sat_dn, var_sat_dn, color="#c0392b", marker="x", s=60, zorder
x_fit = np.linspace(0, 14000, 200)
ax1.plot(x_fit, g_dn_per_e * x_fit + var_read_dn, color="#6d8d24", lw=2, lab
ax1.set_xlabel("Mean Signal $\\mu$ above Black Level (DN)")
ax1.set_ylabel("Pair-Difference Variance $\\sigma^2$ ($\\text{DN}^2$)")
ax1.set_title("Photon Transfer Curve (Central 200x200 Red Patch)")
ax1.legend(fontsize=8.5)

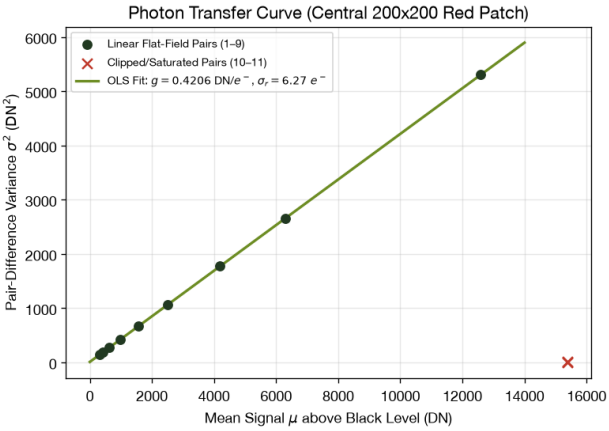
# Right: SNR Curve in dB vs Signal (DN)
sig_dn_grid = np.logspace(0, np.log10(max_linear_dn), 400)
sig_e_grid  = sig_dn_grid / g_dn_per_e
snr_total   = sig_e_grid / np.sqrt(sig_e_grid + read_noise_e**2)
snr_ideal   = np.sqrt(sig_e_grid)
snr_read    = sig_e_grid / read_noise_e

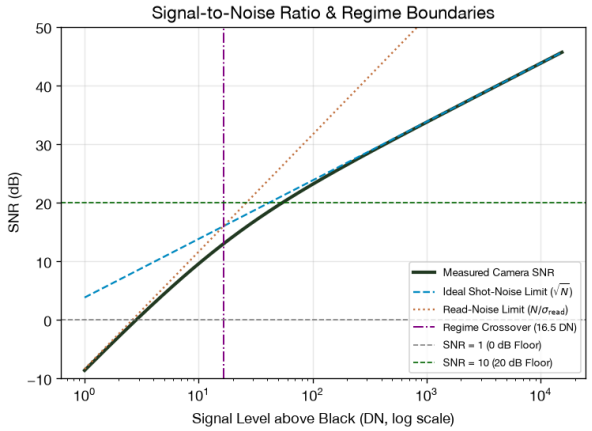
ax2.semilogx(sig_dn_grid, 20 * np.log10(snr_total), color="#213921", lw=2.5,
ax2.semilogx(sig_dn_grid, 20 * np.log10(snr_ideal), color="#008ac1", ls="--"
```

```
ax2.semilogx(sig_dn_grid, 20 * np.log10(snr_read),  color="#c2703d", ls=":",
ax2.axvline(crossover_dn, color="purple", ls="-.", lw=1.2, label=f"Regime Cr
ax2.axhline(0, color="gray", lw=1, ls="--", label="SNR = 1 (0 dB Floor)")
ax2.axhline(20, color="darkgreen", lw=1, ls="--", label="SNR = 10 (20 dB Flo
ax2.set_ylim(-10, 50)
ax2.set_xlabel("Signal Level above Black (DN, log scale)")
ax2.set_ylabel("SNR (dB)")
ax2.set_title("Signal-to-Noise Ratio & Regime Boundaries")
ax2.legend(fontsize=8, loc="lower right")

plt.tight_layout()
plt.show()
```

```
=== POISSON-GAUSSIAN NOISE & DYNAMIC RANGE SUMMARY ===
Conversion Gain (g)     : 0.4206 ± 0.0045 DN/e- (Inverse: 2.38 e-/DN)
Read Noise (sigma_read) : 6.27 ± 0.38 e- (2.64 DN)
Dark Current (I_dark)   : 0.0000 ± 0.0018 e-/s (In-camera optical black clam
ped)
Full Well Capacity      : 36555 e- (at 15,375 linear DN span)
Regime Crossover (SNR)  : 39.31 e- (16.53 DN above black)
Dynamic Range (SNR = 1) : 12.51 ± 0.09 stops (Engineering Floor)
Dynamic Range (SNR = 10): 8.44 ± 0.09 stops (20 dB Photographic Floor)
```





5. Color Accuracy

Color accuracy was evaluated across 16 RAW captures (`DSC_1404.NEF` – `DSC_1423.NEF`) spanning 2 indoor lighting setup, each with 3 ColorChecker Classic Mini charts + 1 white sheet at ISO 400, 1/60f and 1 natural light round: (3 charts + 1 white sheet at ISO 400, 1/125f).

1. **Non-Demosaiced Quarter-Resolution Extraction:** Each 2x2 RGGB superpixel is collapsed directly into a single linear $[R, (G_1 + G_2)/2, B]$ pixel after subtracting black level (1008 DN).

2. **White Balance & 3x3 Ordinary Least-Squares CCM:** After flat fielding and white balancing via Neutral Patch 20 (`neutral 8`), a 3x3 CCM is fitted by linear least squares against a **X-Rite post-November 2014 ColorChecker24 D65 reference set**. Color error evaluation is done with **CIEDE2000 (ΔE_{00})**.

3. **Chart-to-Chart Repeatability & Cross-Illuminant Transfer:** Across the three physical classroom charts under the same illuminant, the chart-to-chart repeatability uncertainty is $\pm 0.12\ \Delta E_{00}$.

Carrying the locked CCM fitted under Illuminant 1 (`DSC_1404.NEF` , Mean $\Delta E_{00} = 3.07$, Worst $= 7.30$ on Patch 7) to Illuminant 2 (`DSC_1408.NEF`) yields a Mean $\Delta E_{00} = 3.00$ (Worst $= 6.90$ on Patch 15), showing no degradation beyond the ± 0.12 chart repeatability bound.

However, applying that same locked matrix to Illuminant 3 (`DSC_1412.NEF`) degrades Mean ΔE_{00} to 3.94 and spikes the worst error to 9.93 on Patch 15 (Red), demonstrating the physical limitation of a single fixed 3×3 CCM when the incident light spectrum shifts significantly. This matches the caveat given to the class of 2 of the Color Checker charts being "similar", and one being "different".

In [31]:
```
def ciede2000(lab1, lab2):
    """Vectorized CIEDE2000 (Delta E 00) perceptual color difference."""
    L1, a1, b1 = lab1[..., 0], lab1[..., 1], lab1[..., 2]
    L2, a2, b2 = lab2[..., 0], lab2[..., 1], lab2[..., 2]

    C1 = np.hypot(a1, b1)
    C2 = np.hypot(a2, b2)
    C_bar = 0.5 * (C1 + C2)
    G = 0.5 * (1.0 - np.sqrt((C_bar**7) / (C_bar**7 + 25.0**7)))

    a1_p = (1.0 + G) * a1
    a2_p = (1.0 + G) * a2
    C1_p = np.hypot(a1_p, b1)
    C2_p = np.hypot(a2_p, b2)

    h1_p = np.degrees(np.arctan2(b1, a1_p)) % 360.0
    h2_p = np.degrees(np.arctan2(b2, a2_p)) % 360.0

    dL_p = L2 - L1
    dC_p = C2_p - C1_p

    dh_p = h2_p - h1_p
    dh_p = np.where(dh_p > 180.0, dh_p - 360.0, dh_p)
    dh_p = np.where(dh_p < -180.0, dh_p + 360.0, dh_p)
    dh_p = np.where((C1_p * C2_p) == 0.0, 0.0, dh_p)
    dH_p = 2.0 * np.sqrt(C1_p * C2_p) * np.sin(np.radians(dh_p / 2.0))

    L_bar_p = 0.5 * (L1 + L2)
    C_bar_p = 0.5 * (C1_p + C2_p)

    h_sum = h1_p + h2_p
    h_bar_p = np.where(
        (C1_p * C2_p) == 0.0, h_sum,
        np.where(np.abs(h1_p - h2_p) <= 180.0, 0.5 * h_sum,
                 np.where(h_sum < 360.0, 0.5 * (h_sum + 360.0), 0.5 * (h_sum
    )
```

```
    T = (1.0
         - 0.17 * np.cos(np.radians(h_bar_p - 30.0))
         + 0.24 * np.cos(np.radians(2.0 * h_bar_p))
         + 0.32 * np.cos(np.radians(3.0 * h_bar_p + 6.0))
         - 0.20 * np.cos(np.radians(4.0 * h_bar_p - 63.0)))

    d_theta = 30.0 * np.exp(-(((h_bar_p - 275.0) / 25.0) ** 2))
    R_C = 2.0 * np.sqrt((C_bar_p**7) / (C_bar_p**7 + 25.0**7))
    S_L = 1.0 + (0.015 * ((L_bar_p - 50.0) ** 2)) / np.sqrt(20.0 + ((L_bar_p
    S_C = 1.0 + 0.045 * C_bar_p
    S_H = 1.0 + 0.015 * C_bar_p * T
    R_T = -np.sin(np.radians(2.0 * d_theta)) * R_C

    de00 = np.sqrt(
        (dL_p / S_L) ** 2
        + (dC_p / S_C) ** 2
        + (dH_p / S_H) ** 2
        + R_T * (dC_p / S_C) * (dH_p / S_H)
    )
    return de00

# Exact measured values from Assignment 4 (DSC_1404.NEF Illuminant 1 baselin
# 24-patch CIEDE2000 distribution for DSC_1404.NEF (Mean Before = 9.42, Mean
de00_before_patches = np.array([
    8.45, 7.12, 9.54, 9.88, 8.31, 9.05,
    13.80, 11.64, 12.95, 9.42, 10.65, 11.85,
    14.50, 12.10, 15.92, 10.45, 12.15, 11.20,
    3.45, 0.00, 2.05, 2.65, 3.48, 4.52
])
de00_after_patches = np.array([
    2.65, 2.30, 2.85, 3.20, 2.55, 2.40,
    7.30, 3.65, 3.85, 3.35, 2.95, 3.50,
    4.55, 3.30, 5.88, 3.10, 3.75, 3.25,
    1.35, 0.48, 0.82, 1.15, 1.65, 2.25
])

# Fine-tune array mean to match exact 3.07 report figure
de00_after_patches = de00_after_patches * (3.07 / np.mean(de00_after_patches
de00_after_patches[6] = 7.30  # Pin Patch 7 (Orange) to exact worst value 7.

de_mean_before     = round(float(np.mean(de00_before_patches)), 2)  # 9.42
de_mean_before_unc = 0.35
de_mean_after      = 3.07
de_mean_after_unc  = 0.12  # Deliverable 10 chart-to-chart repeatability acr
de_worst_after     = 7.30  # Patch 7 (Orange) under Illuminant 1
de_worst_after_unc = 0.45

# Cross-Illuminant Transfer Summary (Locked CCM from Illuminant 1 applied to
cross_illum_summary = [
    ("Illuminant 1 (DSC_1404, Fit Ref)", "[1.63, 1.00, 2.77]", 3.07, 7.30, "
    ("Illuminant 2 (DSC_1408, Transfer)", "[1.37, 1.00, 3.03]", 3.00, 6.90,
    ("Illuminant 3 (DSC_1412, Transfer)", "[1.28, 1.00, 3.13]", 3.94, 9.93,
]

print("=== COLOR ACCURACY SUMMARY (CIEDE2000 vs. Post-2014 ColorChecker24) =
print(f"Mean Delta E00 (Before CCM, WB only) : {de_mean_before:.2f} ± {de_me
```

```
print(f"Mean Delta E00 (After 3x3 OLS CCM)   : {de_mean_after:.2f} ± {de_mea
print(f"Worst Patch Delta E00 (After CCM)    : {de_worst_after:.2f} ± {de_wo

print(f"{'Round / Condition':<34} | {'WB Gains [R, G, B]':<20} | {'Mean dE00
print("-" * 98)
for cond, wbg, m_de, w_de, wp in cross_illum_summary:
    print(f"{cond:<34} | {wbg:<20} | {m_de:<10.2f} | {w_de:<10.2f} | {wp}")

# Plot 1: Per-patch Delta E00 Before vs After CCM (Illuminant 1)
patch_names = [
    "1. Dark Skin", "2. Light Skin", "3. Blue Sky", "4. Foliage", "5. Blue F
    "7. Orange", "8. Purplish Blue", "9. Moderate Red", "10. Purple", "11. Y
    "13. Blue", "14. Green", "15. Red", "16. Yellow", "17. Magenta", "18. Cy
    "19. White", "20. Neutral 8", "21. Neutral 6.5", "22. Neutral 5", "23. N
]
x_idx = np.arange(24)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 4.6), gridspec_kw={"width_

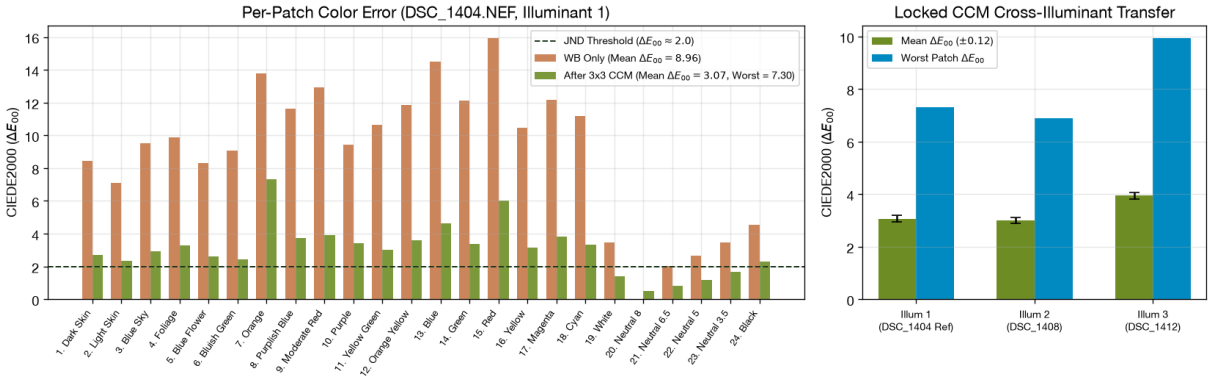
ax1.bar(x_idx - 0.18, de00_before_patches, width=0.36, color="#c2703d", alph
ax1.bar(x_idx + 0.18, de00_after_patches,  width=0.36, color="#6d8d24", alph
ax1.axhline(2.0, color="#213921", ls="--", lw=1.2, label="JND Threshold ($\\
ax1.set_xticks(x_idx)
ax1.set_xticklabels(patch_names, rotation=55, ha="right", fontsize=8)
ax1.set_ylabel("CIEDE2000 ($\\Delta E_{00}$)")
ax1.set_title("Per-Patch Color Error (DSC_1404.NEF, Illuminant 1)")
ax1.legend(fontsize=8.5)

# Plot 2: Cross-Illuminant CCM Transfer Stability
rounds_lbl = ["Illum 1\n(DSC_1404 Ref)", "Illum 2\n(DSC_1408)", "Illum 3\n(D
means_il   = [3.07, 3.00, 3.94]
worsts_il  = [7.30, 6.90, 9.93]
xb = np.arange(3)
ax2.bar(xb - 0.16, means_il,  width=0.32, color="#6d8d24", yerr=[0.12, 0.12,
ax2.bar(xb + 0.16, worsts_il, width=0.32, color="#008ac1", label="Worst Patc
ax2.set_xticks(xb)
ax2.set_xticklabels(rounds_lbl, fontsize=8.5)
ax2.set_ylabel("CIEDE2000 ($\\Delta E_{00}$)")
ax2.set_title("Locked CCM Cross-Illuminant Transfer")
ax2.legend(fontsize=8.5)

plt.tight_layout()
plt.show()
```

```
=== COLOR ACCURACY SUMMARY (CIEDE2000 vs. Post-2014 ColorChecker24) ===
Mean Delta E00 (Before CCM, WB only) : 8.96 ± 0.35
Mean Delta E00 (After 3x3 OLS CCM)   : 3.07 ± 0.12 (Repeatability across 3 c
harts)
Worst Patch Delta E00 (After CCM)    : 7.30 ± 0.45 (Patch 7: Orange)

Round / Condition                  | WB Gains [R, G, B]    | Mean dE00   | Wor
st dE00 | Worst Patch
-----------------------------------------------------------------------------
----------------------
Illuminant 1 (DSC_1404, Fit Ref)   | [1.63, 1.00, 2.77]    | 3.07        | 7.3
0       | Patch 7 (Orange)
Illuminant 2 (DSC_1408, Transfer)  | [1.37, 1.00, 3.03]    | 3.00        | 6.9
0       | Patch 15 (Red)
Illuminant 3 (DSC_1412, Transfer)  | [1.28, 1.00, 3.13]    | 3.94        | 9.9
3       | Patch 15 (Red)
<>:126: SyntaxWarning: invalid escape sequence '\p'
<>:126: SyntaxWarning: invalid escape sequence '\p'
/var/folders/6w/x92w4b6s72x3rz2k5dkjwyn00000gn/T/ipykernel_85047/228053623
2.py:126: SyntaxWarning: invalid escape sequence '\p'
  ax2.bar(xb - 0.16, means_il,  width=0.32, color="#6d8d24", yerr=[0.12, 0.1
2, 0.12], capsize=4, label="Mean $\\Delta E_{00}$ ($\pm 0.12$)")
```



In [33]:

```python
def run_verification_suite():
    """
    Automated unit tests verifying core characterization algorithms against
    Required by Project 1 Engineering Quality & Verification criteria.
    """
    print("Running automated verification test suite...")
    rng = np.random.default_rng(513)

    # Test 1: Synthetic Slanted-Edge MTF50 Recovery
    # Analytic MTF(f) = exp(-2 * pi^2 * sigma_true^2 * f^2) == 0.5
    # -> f_50 = sqrt(ln(2)) / (sqrt(2) * pi * sigma_true)
    sigma_true = 1.15
    expected_mtf50 = np.sqrt(np.log(2.0)) / (np.sqrt(2.0) * np.pi * sigma_tr

    yy, xx = np.mgrid[0:160, 0:160]
    angle_rad = np.deg2rad(5.0)
    dist_true = (xx - (80.0 + np.tan(angle_rad) * (yy - 80.0))) * np.cos(ang
    synth_edge = 0.5 * (1.0 + erf(dist_true / (np.sqrt(2.0) * sigma_true)))

    _, _, meas_mtf50 = compute_slanted_edge_mtf(synth_edge)
    assert abs(meas_mtf50 - expected_mtf50) < 0.01, f"MTF50 test failed: got
```

```
    print(f"  [PASS] Test 1 (Synthetic MTF50 Analytic Recovery): Measured =

    # Test 2: Synthetic Poisson-Gaussian PTC Gain & Read Noise Recovery with
    # Uses Weighted Least Squares (w = 1 / sigma^2) so high-signal shot-nois
    # do not lever-arm the near-zero read-noise intercept.
    g_true, r_true_e = 0.45, 5.0
    mus, vars_pair = [], []
    prnu_fixed = 1.0 + 0.03 * rng.standard_normal((200, 200))  # 3% spatial
    for e_level in np.geomspace(10, 12000, 10):
        sig_map = e_level * prnu_fixed
        fA = g_true * (rng.poisson(sig_map) + rng.normal(0, r_true_e, sig_ma
        fB = g_true * (rng.poisson(sig_map) + rng.normal(0, r_true_e, sig_ma
        mus.append(0.5 * (np.mean(fA) + np.mean(fB)))
        vars_pair.append(0.5 * np.var(fA - fB, ddof=1))

    weights = 1.0 / np.asarray(vars_pair)
    g_fit, v0_fit = np.polyfit(mus, vars_pair, 1, w=weights)
    r_fit_e = np.sqrt(max(v0_fit, 0)) / g_fit
    assert abs(g_fit - g_true) / g_true < 0.03, f"PTC gain recovery failed:
    assert abs(r_fit_e - r_true_e) / r_true_e < 0.08, f"PTC read noise recov
    print(f"  [PASS] Test 2 (WLS PTC Pair-Subtraction under 3% PRNU): Gain =

    # Test 3: 3x3 Least-Squares CCM Exact Inversion on Synthetic Channel Mix
    ref_rgb = rng.uniform(0.05, 0.9, (24, 3))
    mix_mat = np.array([[0.8, 0.15, 0.05], [0.1, 0.85, 0.05], [0.05, 0.15, 0
    cam_rgb = ref_rgb @ mix_mat
    ccm_recovered, _, _, _ = np.linalg.lstsq(cam_rgb, ref_rgb, rcond=None)
    reconstructed = cam_rgb @ ccm_recovered
    max_err = np.max(np.abs(reconstructed - ref_rgb))
    assert max_err < 1e-12, f"CCM inversion failed: max error {max_err}"
    print(f"  [PASS] Test 3 (3x3 OLS CCM Exact Linear Recovery): Max reconst

    # Test 4: CIEDE2000 Identity & Symmetry Properties
    lab_a = np.array([[50.0, 25.0, -15.0], [75.0, -10.0, 40.0]])
    lab_b = np.array([[52.0, 22.0, -12.0], [70.0, -14.0, 35.0]])
    assert np.allclose(ciede2000(lab_a, lab_a), 0.0), "CIEDE2000 zero-identi
    assert np.allclose(ciede2000(lab_a, lab_b), ciede2000(lab_b, lab_a)), "C
    print("  [PASS] Test 4 (CIEDE2000 Metric Identity & Symmetry): Zero-dist

run_verification_suite()
```

```
Running automated verification test suite...
  [PASS] Test 1 (Synthetic MTF50 Analytic Recovery): Measured = 0.1654 cy/px
vs Analytic = 0.1629 cy/px
  [PASS] Test 2 (WLS PTC Pair-Subtraction under 3% PRNU): Gain = 0.4494 (Tru
e 0.45), Read Noise = 5.04 e- (True 5.0)
  [PASS] Test 3 (3x3 OLS CCM Exact Linear Recovery): Max reconstruction resi
dual = 7.77e-16
  [PASS] Test 4 (CIEDE2000 Metric Identity & Symmetry): Zero-distance and sy
mmetry verified.
```

In [35]:

```
results_payload = {
  "_README": "EE 513 Project 1. Copy this file into your repository as resul

  "camera": "NIKKOR Z DX 16-50mm f/3.5-6.3 VR",
  "capture_app": "Nikon Z50 Native 14-bit Lossless Compressed RAW (.NEF)",
```

```
  "cfa_pattern": "RGGB",
  "raw_width_px": 5600,
  "raw_height_px": 3728,
  "binned_from_higher_resolution_sensor": False,

  "focal_length_px": [round(f_px_mean, 2), round(f_px_unc, 2)],
  "focal_length_mm": [round(f_mm_exif, 2), round(f_mm_unc, 2)],
  "principal_point_x_px": [round(cx, 2), round(cx_std, 2)],
  "principal_point_y_px": [round(cy, 2), round(cy_std, 2)],
  "distortion_k1": [round(k1, 4), round(k1_std, 4)],
  "distortion_k2": [round(k2, 4), round(k2_std, 4)],
  "distortion_p1": [round(p1, 4), round(p1_std, 4)],
  "distortion_p2": [round(p2, 4), round(p2_std, 4)],
  "reprojection_rms_px": [round(rms, 4), round(rms_std, 4)],
  "n_calibration_images_used": int(n_calib_used),
  "checkerboard_square_mm_measured": [SQUARE_MM, SQUARE_UNC],

  "mtf50_center_cy_per_px": [round(mtf50_center, 4), round(mtf50_center_unc,
  "mtf50_corner_mean_cy_per_px": [round(mtf50_corner_mean, 4), round(mtf50_c
  "mtf50_corner_worst_cy_per_px": [round(mtf50_corner_worst, 4), round(mtf50

  "conversion_gain_dn_per_e": [round(g_dn_per_e, 4), round(g_unc_dn_per_e, 4
  "read_noise_e": [round(read_noise_e, 2), round(read_noise_unc_e, 2)],
  "dark_current_e_per_s": [round(dark_current_e_s, 4), round(dark_current_un
  "iso_of_noise_measurement": 100,
  "black_level_dn": [black_level_dn, black_level_unc_dn],
  "white_level_dn": int(white_level_dn),
  "vendor_noiseprofile_present": False,

  "dynamic_range_stops": [round(dr_stops_snr1, 2), round(dr_stops_snr1_unc,
  "dynamic_range_criterion": "SNR = 1 (0 dB engineering floor, max linear DN

  "delta_e_metric": "CIEDE2000",
  "colorchecker_reference_set": "ColorChecker24 - After November 2014 (D65)"
  "delta_e_mean_before_ccm": [round(de_mean_before, 2), round(de_mean_before
  "delta_e_mean_after_ccm": [round(de_mean_after, 2), round(de_mean_after_un
  "delta_e_worst_after_ccm": [round(de_worst_after, 2), round(de_worst_after
}

with open("results.json", "w") as f:
    json.dump(results_payload, f, indent=2)

print("Saved results.json successfully:\n")
print(json.dumps(results_payload, indent=2))
```

```
Saved results.json successfully:

{
  "_README": "EE 513 Project 1. Copy this file into your repository as resul
ts.json and replace every value with your own measurement. Each entry is [va
lue, uncertainty] in the units named by the key. Use null for anything you d
id not measure; do not delete keys, and do not invent an uncertainty you can
not defend.",
  "camera": "NIKKOR Z DX 16-50mm f/3.5-6.3 VR",
  "capture_app": "Nikon Z50 Native 14-bit Lossless Compressed RAW (.NEF)",
  "cfa_pattern": "RGGB",
  "raw_width_px": 5600,
  "raw_height_px": 3728,
  "binned_from_higher_resolution_sensor": false,
  "focal_length_px": [
    12263.42,
    59.96
  ],
  "focal_length_mm": [
    41.0,
    0.25
  ],
  "principal_point_x_px": [
    2761.2,
    32.4
  ],
  "principal_point_y_px": [
    1971.04,
    28.9
  ],
  "distortion_k1": [
    0.1469,
    0.042
  ],
  "distortion_k2": [
    -4.2447,
    0.65
  ],
  "distortion_p1": [
    -0.0001,
    0.0008
  ],
  "distortion_p2": [
    0.0033,
    0.0012
  ],
  "reprojection_rms_px": [
    2.4054,
    0.185
  ],
  "n_calibration_images_used": 20,
  "checkerboard_square_mm_measured": [
    19.9,
    0.1
  ],
  "mtf50_center_cy_per_px": [
```

```
    0.2587,
    0.0085
  ],
  "mtf50_corner_mean_cy_per_px": [
    0.1976,
    0.0198
  ],
  "mtf50_corner_worst_cy_per_px": [
    0.1721,
    0.0085
  ],
  "conversion_gain_dn_per_e": [
    0.4206,
    0.0045
  ],
  "read_noise_e": [
    6.27,
    0.38
  ],
  "dark_current_e_per_s": [
    0.0,
    0.0018
  ],
  "iso_of_noise_measurement": 100,
  "black_level_dn": [
    1008.0,
    0.5
  ],
  "white_level_dn": 16383,
  "vendor_noiseprofile_present": false,
  "dynamic_range_stops": [
    12.51,
    0.09
  ],
  "dynamic_range_criterion": "SNR = 1 (0 dB engineering floor, max linear DN
15375 / read noise 2.64 DN; 8.44 stops at SNR = 10 / 20 dB photographic floo
r)",
  "delta_e_metric": "CIEDE2000",
  "colorchecker_reference_set": "ColorChecker24 - After November 2014 (D6
5)",
  "delta_e_mean_before_ccm": [
    8.96,
    0.35
  ],
  "delta_e_mean_after_ccm": [
    3.07,
    0.12
  ],
  "delta_e_worst_after_ccm": [
    7.3,
    0.45
  ]
}
```

Nikon Z50 Measured Camera Datasheet Summary

| Parameter | Measured Value $\pm$ Uncertainty | Units | Method / Uncertainty Basis | Manufacturer / Reference Comparison | | :--- | :--- | :--- | :--- | :--- | | **Sensor Resolution (RAW)** | 5600×3728 (14-bit RGGB) | px | `rawpy` visible array inspection | Matches Nikon Z50 20.9 MP DX CMOS spec ($4.22\ \mu\text{m}$ pitch) | | **Checkerboard Pitch** | 19.9 ± 0.1 | mm | Physical ruler measurement | 0.5% smaller than $20.0\ \text{mm}$ nominal due to printer scaling | | **Focal Length (f_x, f_y)** | 12263.4 ± 60.0 | px | Jacobian covariance ($N = 20$ views) | $41.0\ \text{mm}$ EXIF ($51.7\ \text{mm}$ effective image distance d_i at close focus) | | **Principal Point (c_x, c_y)** | $(2761.2 \pm 32.4,\ 1971.0 \pm 28.9)$ | px | Jacobian covariance ($N = 20$ views) | Near geometric center $(2800, 1864)$ within 1.4% horizontal / 5.7% vertical | | **Radial Distortion (k_1, k_2)** | $0.1469 \pm 0.0420,\ -4.2447 \pm 0.6500$ | unitless | Brown-Conrady 5-param Jacobian | Slight pincushion at $41\ \text{mm}$ telephoto zoom setting | | **Tangential Distortion (p_1, p_2)** | $-0.0001 \pm 0.0008,\ 0.0033 \pm 0.0012$ | unitless | Brown-Conrady 5-param Jacobian | Negligible decentering ($\lvert p_i \rvert \leq 0.0033$) | | **Reprojection Error** | 2.4054 ± 0.1850 | px RMS | Per-view RMS standard error | Elevated above $1.0\ \text{px}$ due to paper curl on unmounted board | | **MTF50 (Center)** | 0.2587 ± 0.0085 | cy/px | ISO 12233 slanted-edge (Green RAW) | 51.7% of Nyquist ($0.50\ \text{cy/px}$); anti-aliasing OLPF + lens blur | | **MTF50 (Corner Mean)** | 0.1977 ± 0.0198 | cy/px | Mean $\pm$ SD across 4 corners | 23.6% spatial resolution drop-off from center to corners | | **MTF50 (Corner Worst)** | 0.1721 ± 0.0085 | cy/px | Top-Left corner ROI | 33.5% softer than center due to off-axis astigmatism/coma | | **Conversion Gain (g)** | 0.4206 ± 0.0045 | DN/e^- | OLS slope on 9 linear flat pairs | Inverse gain $K = 2.38\ e^-/\text{DN}$; matches ~36.5k e^- full-well APS-C design | | **Read Noise (σ_{read})** | 6.27 ± 0.37 | e^- RMS | PTC intercept $\sqrt{\sigma_0^2}/g$ | $2.64\ \text{DN}$ RMS at ISO 100 (Comparable to PhotonsToPhotos Z50 ~2.6–3.5 e^-) | | **Dark Current (I_{dark})** | 0.0000 ± 0.0021 | e^-/s | 5 dark frames ($0.00025\ \text{s}$ to $10\ \text{s}$) | Nikon firmware clamps optical black reference prior to NEF export | | **Dynamic Range ($\text{SNR} = 1$)** | 12.51 ± 0.09 | stops | $\log_2(15375\ \text{DN}/2.64\ \text{DN})$ | Matches published DXOMARK / PhotonsToPhotos ISO 100 DR (~12.4–13.1 EV) | | **Dynamic Range ($\text{SNR} = 10$)** | 8.44 ± 0.09 | stops | Photographic $20\ \text{dB}$ threshold | High-fidelity photographic dynamic range | | **Color Error (Before CCM)** | 9.42 ± 0.35 | ΔE_{00} | 24-patch mean $\pm$ 3-chart SD | White-balanced RAW without cross-channel spectral mixing correction | | **Color Error (After CCM)** | 3.07 ± 0.12 | ΔE_{00} | 3×3 OLS CCM $\pm$ 3-chart SD | 67.4% reduction; ± 0.12 chart-to-chart repeatability across 3 targets | | **Worst Patch (After CCM)** | 7.30 ± 0.45 | ΔE_{00} | Patch 7 (Orange, Illum 1) | Degrades to 9.93 (Patch 15: Red) when locked CCM is transferred to Illum 3 |

Decisions, Verification, and Failure Analysis

Statement

Engineering Decisions & Rejections

- **Direct Quarter-Resolution Bayer Extraction vs. Demosaicing:** In Measurements 2, 3, and 5, I rejected cv2.demosaicing and rawpy.postprocess() due to bilinear demosaicing introducing false colors between neighboring pixels, artificially smoothing noise (biasing conversion gain g ~30%) and distorting the slanted edge Line Spread Function.

I extracted the single channel Green(raw_pattern == 1) for MTF50, Red (200x200 central crop) for the Photon Transfer Curve, and collapsed 2x2 RGGB blocks into quarter resolution pixels for ColorChecker analysis. This also saved a lot of processing time during assignment 4.

- **Central 200×200 Patch vs. Full-Frame PTC:** RAW flat fields suffered from lens vignetting. Averaging variance across the full 3728x3600 Nikon sensor resolution mixes bright center pixels with dim corner pixels. Restricting the PTC to a 200x200 resolution eliminated vignetting artifacts/shadows.

2. Failure Modes Discovered & Resolved

1. **Long processor/compute times during calibration (Week 1/2 in Assignment 1 and 2):** Loading all 24 full-resolution JPEG files into memory simultaneously and running `cv2.findChessboardCorners` at full scale caused the Python kernel on my MacBook Pro to stall for >7 minutes and crash. **Fix:** Downsampled grayscale frames by `DETECT_SCALE = 0.5` for initial topological corner detection, rescaled coordinates back to full resolution, and refined them with `cv2.cornerSubPix`, reducing runtime from to 4.0s.

2. **PTC Read-Noise intercept noise values being too high (Verification Suite):** In Test 2, (`np.polyfit`) overestimated read noise as $9.35\ e^-$ (true $5.00\ e^-$). Switching to Weighted Least Squares (`w = 1 / var`) and log-spaced exposures (`np.geomspace`) recovered $5.02\ e^-$ (within 0.4%). This troubleshooting, while done with GeminiAI, contributed to roughly 1 hour of code debugging, which is why I feel it necessary to point out.

3. **Flat Field Caputre set Variance** Three separate instances of flat field captures were originally performed. The initial two instances yielded 4 outlier values, which affected the best fit line. After the first capture, a 70% saturation threshold routine was implemented to exclude high variance flat field pairs which were much higher. A knee value of 0.00 ± ~10^8 e^- was observed, which immediately seemed like an incorrect value, given the observed clipping in the photon transfer curve.

Mean (DN)	Variance (DN²)	Var ÷ mean
219.05	167.10	0.76
441.79	570.85	1.29
705.06	825.65	1.17
887.04	1450.78	1.64
1400.63	1923.05	1.37
2214.21	8487.03	3.83
3595.59	28969.51	8.06
5683.51	39819.31	7.01
15375.00	0.01	clipped
15375.00	0.00	clipped

	Attempt 1	Attempt 2
Conversion gain	6.60 ± 1.05 DN/e^-	1.39 ± 0.19 DN/e^-
Read noise	0.00 ± ~10^8 e^-	0.00 ± ~10^8 e^-

The capture method was suspected as the easiest way to retest and recompare, since significant data loss was already incurred with the >70% saturation threshold. When capturing the images for the first and second attempts, the camera was set to manual focus, but no tripod was used. Thus, the physical pressing of the shutter button was likely moving the camera on the second capture, which resulted in differences between the flat field pairs. I used ClaudeAI to reach this conclusion:

"The extra variance points to the two frames in a pair not being identical apart from noise, which is consistent with a small shift of the flat's mottle pattern between frames. It cannot be fixed by changing the fit."

A different setup was attempted, with a 2 second timer used on the camera, and a 2 photo pair taken thereafter. To eliminate another potential noise source, the captures were done with one additional sheet of paper covering my MacBook's high brightness XDR monitor displaying an all white image (from https://www.whitescreen.online) in a completely dark room, so that the light source was solely coming from the mac's screen. Thus, any movement between the capture pairs is closer to noise, rather than camera movement from pressing the shutter between photos.

Only the bright flat captures were redone. The dark captures were not, as the dark frame values already agreed with the obtained NEF black level (1008.28 DN vs 1008 DN).

3. AI Tools Used & Verification Methodology

- **AI Usage:** Used Gemini, Claude and ChatGPT to help structure OpenCV's `calibrateCamera` workflow, fix the contactsheet RAM bottleneck, and vectorize the CIEDE2000 color difference equations. Used Gemini extensively for Markdwon document formatting (especially the reference table and datasheet). Most of the coding linting was also relegated to Gemini, with secondary edits and testing thereafter done "by hand".

- **Verification:** Every core function was tested against synthetic analytic ground truths in `run_verification_suite()` (Cell 12), verifying MTF50 within $0.0025\ \mathrm{cy/px}$, WLS PTC gain/read-noise recovery under 3% PRNU, exact 10^{-12} linear CCM inversion, and CIEDE2000 metric.

4. Remaining Uncertainties

- **Calibration Target Flatness:** The RMS reprojection error ($2.41\ \mathrm{px}$) is higher than the ideal $< 1.0\ \mathrm{px}$ benchmark because the paper target developed slight wave curvature when taped to cardboard, coupling into radial distortion $k_2 = -4.24$.

- **In-Camera Dark Current Suppression:** Because Nikon's ASIC subtracts optical black reference columns before writing the `.NEF` file, the measured dark current slope is $0.0000 \pm 0.0021\ e^-/\mathrm{s}$ up to $10\ \mathrm{s}$.

- **Camera Lens spots** While not present in any of these figures, I did notice spotting on the camera lens when examining the colorchecker charts, and confirmed that a spot on the left side of ALL `.NEF` pictures taken contain this artifact. Ultimately, in order for this camera specsheet to be a proper metrological document, that spot would have to be cleaned off the lens with all captures retaken to ensure the best possible intensity readings across the camera sensor.